

1. Optimum Length of Window in Moving Mean

Better NOT to have a big window in moving mean method as averaging across a wide horizon

a) reduces variability

b) But smoothens the noise

c) Smoothing out may not be bad but increasing window by too much leads to over smoothing of the data, destroying real signals and trends

d) May lead to infeasible estimates in imputation of missing values

1. Optimum Length of Window in Moving Mean

Summary of Window Sizes

Window Size	Advantages	Disadvantages
Small Window	Retains local trends, adapts quickly to sudden shifts.	Can be overly sensitive to random noise/outliers.
Large Window	Effectively eliminates noise, very smooth.	Laggy; introduces irrelevant historical data; yields infeasible estimates for local missing values.

When we talk about "**introducing irrelevant historical data**" in the context of a moving average (or any rolling time-series method), we mean that the mathematical window has grown so large that it starts pulling in past data points that **no longer reflect the current reality or trend** of the system.

A moving mean assumes that data points close to each other in time are similar. If you use a very wide window, you end up averaging data from completely different cyclical patterns, seasons, or structural states.

2. What is then Optimum Window Length in Moving Mean?

- ❖ Not too big as you may over smooth the data destroying real signals and trends and misses local variations
- ❖ Not too small as data may be sensitive to effects of noise
- ❖ Not possible to give a definitive window size and depends upon the context and
- ❖ whether data is rather static (larger window to reduce noise effects) or very dynamic (small window to capture fast changing local effects so that the mean predicted value does not lag behind)

3. Even Moving Mean – Matlab Code

```
% Sample data containing missing values (NaN)
data = [10, 12, NaN, 14, 15, NaN, NaN, 18, 20, 22];
% 1. Impute missing data using a moving mean
(window size of 4)
filledMean = fillmissing(data, 'movmean', 4);
% 2. Impute missing data using a moving median
(window size of 4)
filledMedian = fillmissing(data, 'movmedian', 4);
```

```
% Display results
disp('Original Data:');
disp(data);
disp('Filled Data (Moving Mean):');
disp(filledMean);
disp('Filled Data (Moving Median):');
disp(filledMedian);
```

```
Original Data:
    10    12   NaN    14    15   NaN   NaN    18    20    22

Filled Data (Moving Mean):
    10.0000    12.0000    12.0000    14.0000    15.0000    14.5000    16.5000    18.0000    20.0000    22.0000

Filled Data (Moving Median):
    10.0000    12.0000    12.0000    14.0000    15.0000    14.5000    16.5000    18.0000    20.0000    22.0000
```

3. Even Moving Mean – Matlab Code

```
% Sample data containing missing values (NaN)
data = [10, 12, NaN, 14, 15, NaN, NaN, 18, 20, 22];
% 1. Impute missing data using a moving mean (window size of
4)
filledMean = fillmissing(data, 'movmean', 4);
% 2. Impute missing data using a moving median (window size
of 4)
filledMedian = fillmissing(data, 'movmedian', 4);
% Display results
disp('Original Data:');
disp(data);
disp('Filled Data (Moving Mean):');
disp(filledMean);
disp('Filled Data (Moving Median):');
disp(filledMedian);
% Dataset with a rapid value spike and missing values
newData = [5, 6, NaN, 25, 30, NaN, 12, 10];
% Impute missing data using a moving mean of window size 4
filledMean = fillmissing(newData, 'movmean', 4);
filledMedian = fillmissing(newData, 'movmedian', 4);
% Display the results
disp('Original New Data:');
disp(newData);
disp('Filled Data (Moving Mean):');
disp(filledMean);
disp('Filled Data (Moving Median):');
disp(filledMedian);
```

3. Even Moving Mean – Matlab Code Results

Original Data:

10 12 NaN 14 15 NaN NaN 18 20 22

Filled Data (Moving Mean):

10.0000 12.0000 12.0000 14.0000 15.0000 14.5000 16.5000 18.0000 20.0000 22.0000

Filled Data (Moving Median):

10.0000 12.0000 12.0000 14.0000 15.0000 14.5000 16.5000 18.0000 20.0000 22.0000

Original New Data:

5 6 NaN 25 30 NaN 12 10

Filled Data (Moving Mean):

5.0000 6.0000 12.0000 25.0000 30.0000 22.3333 12.0000 10.0000

Filled Data (Moving Median):

5 6 6 25 30 25 12 10

3. Even Moving Mean – Python Code

```
import numpy as np
import pandas as pd
data = pd.Series([10, 12, np.nan, 14, 15, np.nan, np.nan, 18, 20,
22])
filledMean = data.fillna(data.rolling(window=4, min_periods=1,
center=True).mean())
filledMedian = data.fillna(data.rolling(window=4, min_periods=1,
center=True).median())
print('Original Data:')
print(data.to_list())
print('\nFilled Data (Moving Mean):')
print(filledMean.to_list())
print('\nFilled Data (Moving Median):')
print(filledMedian.to_list())
print('-' * 40)
```

```
# Dataset with a rapid value spike and missing values
newData = pd.Series([5, 6, np.nan, 25, 30, np.nan, 12,
10])
filledMeanNew =
newData.fillna(newData.rolling(window=4,
min_periods=1, center=True).mean())
filledMedianNew =
newData.fillna(newData.rolling(window=4,
min_periods=1, center=True).median())

# Display the results
print('\nOriginal New Data:')
print(newData.to_list())
print('\nFilled Data (Moving Mean):')
print(filledMeanNew.to_list())
```

```
print('\nFilled Data (Moving Median):')
print(filledMedianNew.to_list())
```

3. Even Moving Mean – Python Output

```
Original Data:
[10.0, 12.0, nan, 14.0, 15.0, nan, nan, 18.0, 20.0, 22.0]

Filled Data (Moving Mean):
[10.0, 12.0, 12.0, 14.0, 15.0, 14.5, 16.5, 18.0, 20.0, 22.0]

Filled Data (Moving Median):
[10.0, 12.0, 12.0, 14.0, 15.0, 14.5, 16.5, 18.0, 20.0, 22.0]
-----

Original New Data:
[5.0, 6.0, nan, 25.0, 30.0, nan, 12.0, 10.0]

Filled Data (Moving Mean):
[ 5.    6.   12.   25.   30.   22.33 12.   10.  ]

Filled Data (Moving Median):
[5.0, 6.0, 6.0, 25.0, 30.0, 25.0, 12.0, 10.0]
```